
SeRPO: Segment-level Rubric-based Policy Optimization for LLM Agents

MinJeong Ban¹ San Kwon¹

Abstract

Reinforcement learning for long-horizon LLM agents is bottlenecked by its reward signal: a binary, trajectory-level reward is almost always zero and gives no gradient when successful rollouts are rare. Prior remedies either propagate the outcome to individual steps or replace the binary signal with richer rubric scores, but step-level credit still rests on the outcome and rubric scores are applied to the whole trajectory. We propose **SeRPO** (Segment-level Rubric-based Policy Optimization), in which a single frozen-LLM call jointly partitions a trajectory into *subgoal-aligned segments* and scores each segment’s contribution, feeding the segment-level advantages to GRPO with no separately trained reward model. On AppWorld, SeRPO outperforms outcome-reward baselines at the trajectory level and at the step level. Notably, step-level credit raises per-task completion but leaves scenario completion unchanged from the trajectory baselines, whereas SeRPO improves both. And because the segment score is independent of the outcome, SeRPO yields a learning signal even when every rollout fails: trained on failed trajectories alone, where binary and step-level rewards give no gradient at all, it still improves over base.

1. Introduction

Reinforcement learning has become the engine behind today’s most capable LLM agents. Triggered by the success of DeepSeek-R1 (DeepSeek-AI, 2025), a wave of work now uses RL to sharpen the reasoning and decision-making of LLM-based agents, and the benchmarks have grown correspondingly long and intricate. In environments such as AppWorld (Trivedi et al., 2024) and DeepPlanning (Zhang et al., 2026), an agent must navigate many applications through API calls, resolve cross-app references, authenticate, and produce precisely formatted outputs across several steps. In AppWorld, for instance, an agent operates 9 simulated apps through 457 APIs and is graded by stateful unit tests, and even strong proprietary models solve few tasks.

Whether RL can actually train an agent on such tasks comes

down to one thing: credit assignment, the problem of attributing reward to the actions that earned it. On long horizons this is precisely where existing reward designs break down, and they break in two complementary ways. The first failure is *sparsity*. The conventional signal is a binary, trajectory-level reward: a rollout that passes every unit test receives one, and every other rollout receives zero. When complete successes are rare, almost every rollout returns zero, the policy gradient vanishes, and any genuinely good behavior buried inside a failed rollout earns no credit at all. The second failure is *uniform credit*. The natural fix for sparsity is to make the reward denser, for example by exposing AppWorld’s fractional test-pass ratio so that a partial success scores partially, or by replacing the binary signal with a single rubric score. Yet a denser reward computed once for the whole trajectory is still broadcast uniformly to every token. A rollout that decomposes its goal correctly, gathers all the required data, and stumbles only on a final formatting step is then credited exactly like one that gives up after repeated authentication failures, even though the two deserve opposite gradients.

Viewed this way, prior work occupies two extremes of a granularity spectrum, each with a complementary weakness. Outcome-level rewards (binary or continuous) are computed once per trajectory: they are sparse and credit every token uniformly, too coarse to tell a near-success from a give-up. Step-level credit assignment (e.g., GiGPO (Feng et al., 2025), HiPER (Peng et al., 2026), SALT (Li et al., 2026), HGPO (He et al., 2026)) is fine-grained but noisy: it relies on binary or rule-based signals and, in GiGPO’s case, groups steps by matching the surface text of environment observations, which, in free-form-state environments like AppWorld, groups coincidental or degenerate observations rather than genuine state revisits. We argue that an intermediate granularity, the *subgoal-aligned segment* (a contiguous group of steps sharing one subgoal), can provide consistent, meaningful feedback without the noise of step-level rewards or the sparsity of outcome-level rewards. The segment is fine enough to localize failure within a trajectory, coarse enough to be reliably scored by an LLM judge, and assigned by semantic subgoal rather than by surface-string identity.

We instantiate this intermediate granularity with a single frozen-LLM call that jointly segments a trajectory and scores each segment’s contribution, feeding the resulting

segment-level advantages to GRPO with no separately trained reward model. We call the method SeRPO and evaluate it on AppWorld against outcome-reward baselines at the trajectory level (binary and continuous) and at the step level (GiGPO), together with a trajectory-level ablation of our own segment scores. SeRPO improves over every baseline on both metrics; a controlled ablation attributes the gain to segment-level assignment rather than to a denser reward or to the rubric itself; and because each segment score is independent of the outcome, SeRPO retains a usable signal even when every rollout fails. A further benefit of crediting each segment separately is interpretability: one can point to where along a rollout credit was earned, and where a non-verifiable reward might be exploited.

Our contributions are as follows:

1. **The right granularity, not the right density.** We show that relieving sparsity alone is insufficient: a denser reward that is still computed at the trajectory level barely improves over the binary one. The bottleneck is the *granularity* at which credit is assigned, not the density of the signal.
2. **The subgoal-aligned segment as that granularity.** We propose SeRPO, in which a single frozen-LLM call jointly segments a trajectory and scores each segment’s contribution, and the segment-level advantages are fed to GRPO without a trained reward model. SeRPO outperforms both the trajectory-level outcome baselines and a strong step-level method (GiGPO), and two controlled comparisons isolate why: with the rubric held fixed, the same scores help at the segment level but not at the trajectory level, so the segment rather than the rubric drives the gain; and unlike step-level credit, which improves per-task but not full-scenario completion, the segment improves both.
3. **A learning signal even when every rollout fails.** Because the segment score is independent of the trajectory outcome, SeRPO extracts a gradient where outcome-based rewards cannot. Trained on failed trajectories alone, where binary and step-level rewards provide no signal at all, SeRPO improves over base policy, confirming that bottleneck is reward *structure*, not density.
4. **Segment-level interpretability of reward.** We illustrate, via a case study, that assigning the rubric per segment makes it possible to localize where a non-verifiable reward is earned or potentially exploited, which a single trajectory-level score cannot do.

2. Related Work

2.1. Long-Horizon Agentic Benchmarks

Developing RL-based agent training requires benchmarks that expose the limits of sparse rewards. AppWorld (Trivedi et al., 2024) offers nine applications operable through hundreds of APIs with state-based unit tests; unlike benchmarks with a small set of templated actions, it requires an agent to interact through free-form code generation over a combinatorially open action space, and even strong models leave most challenging tasks unsolved. DeepPlanning (Zhang et al., 2026) targets long-horizon planning under global time and budget constraints, and WebShop (Yao et al., 2022) simulates an e-commerce environment with over a million products. Long multi-step trajectories yield low success rates, making sparse binary rewards inadequate for RL.

2.2. Step-Level Credit Assignment

To relieve the sparsity of trajectory-level rewards on long horizons, one line of work propagates the trajectory reward to individual steps. GiGPO (Feng et al., 2025) estimates relative advantages at two levels, over whole trajectories and over groups of anchor states that share the same preceding observation, enabling critic-free step-level optimization. Its anchor grouping matches the surface text of observations, so in free-form-state environments it tends to group coincidental or degenerate observations rather than genuine state revisits. HiPER (Peng et al., 2026) factorizes the policy into a planner and an executor with hierarchical advantages, improving sample efficiency, but its rewards remain rule-based, with no qualitative assessment of execution. These methods sit at the fine, noisy end of the abstraction spectrum, assigning credit per step but resting on binary or rule-based signals and on surface-level state matching rather than semantic subgoals. Crucially, because the credit is still derived from the outcome, it collapses when no rollout succeeds. On a group of all-failing trajectories the signal carries no gradient, so these methods cannot learn from failures alone.

2.3. Rubric-Based Rewards

A way to obtain a signal that does not vanish on failures is to replace the binary outcome with a rubric score that rates quality directly. Rubrics as Rewards (Gunjal et al., 2025) uses an LLM to score responses against multi-criteria rubrics for on-policy RL and outperforms plain LLM-as-judge baselines. Decomposing quality into explicit criteria is also reported to reduce the format and length-based hacking to which a single judge score is prone, which motivates our use of a rubric rather than a scalar judge. These scores, however, are aggregated at the trajectory level, so all actions in a rollout still receive identical credit. AdaRubric (Ding, 2026) generates task-specific rubrics and scores trajectories

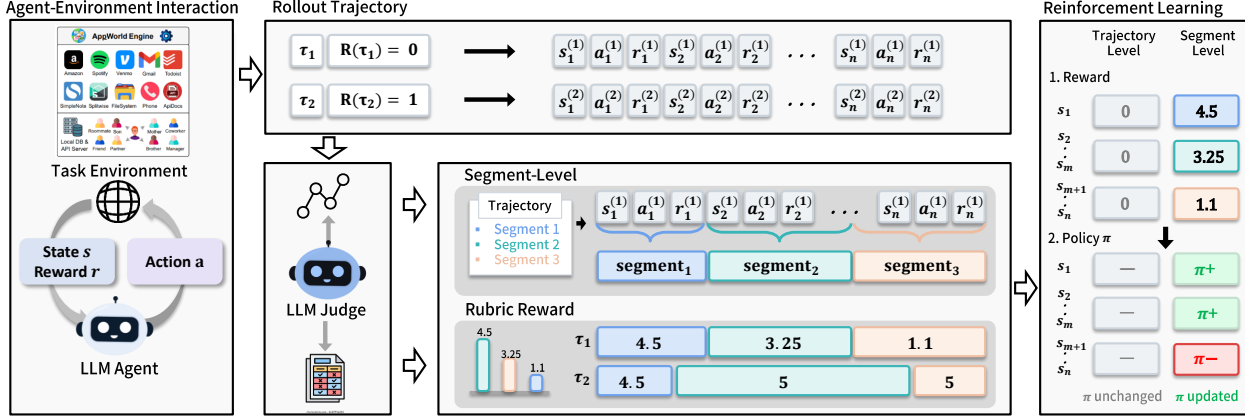


Figure 1. Overview of SeRPO. LLM call jointly partitions a trajectory into subgoal-aligned segments and scores each with a multidimensional rubric. The segment-level scores are used as advantages for a GRPO update, with no separately trained reward model. Unlike a trajectory-level reward, which collapses to zero on a failed rollout ($R(\tau)=0$), each segment is scored independently of the outcome, so SeRPO keeps a usable signal on the correct segments of a failed trajectory while penalizing only the one that caused the failure.

with per-dimension feedback, but targets evaluation rather than RL with segment-level credit. Rubric rewards thus supply multi-dimensional quality but only at the trajectory level, the coarsest end of the abstraction spectrum, leaving the intermediate, subgoal-aligned granularity that SeRPO occupies unexplored.

3. Method

3.1. Problem Setup and Motivation

We consider an LLM agent producing a trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ of T steps, evaluated by an outcome function. Under a binary reward, $R(\tau) \in \{0, 1\}$ indicates whether all unit tests pass; under a continuous reward, $R(\tau) \in [0, 1]$ is the fraction of tests passed. In standard GRPO, the per-token advantage is obtained by normalizing a single trajectory-level reward across a group of rollouts and broadcasting it to every token. Both binary and continuous outcome rewards therefore share a structural property: *every token in a trajectory receives the same credit*, regardless of which phase it belongs to. A “near-miss” trajectory that decomposes the goal correctly, gathers all data, and fails only on output format is credited the same as a “give-up” trajectory that abandons the task after repeated authentication failures, even though the two warrant very different gradients. This is the uniform-credit limitation SeRPO targets.

3.2. SeRPO Overview

SeRPO consists of two stages (See Fig. 1): (1) a single frozen-LLM call that jointly partitions a trajectory into subgoal-aligned segments *and* assigns each segment a contribution score; (2) conversion of those segment scores into per-token advantages for a GRPO update. An optional rubric generator can co-evolve the scoring criteria with the policy.

3.3. Joint Segmentation and Scoring

SeRPO uses a single frozen off-the-shelf LLM call that performs two coupled tasks on a trajectory τ . First, it partitions τ into M contiguous, non-overlapping segments $\mathcal{S} = \{s_1, \dots, s_M\}$, where each segment is a maximal run of steps sharing one subgoal. Second, for each segment it emits a subgoal description, a phase type from a fixed taxonomy (`login`, `api-exploration`, `data-fetch`, `computation`, `action`, `completion`, `error-recovery`), and a set of rubric scores assessing how productively that segment advanced the task in the context of the whole trajectory. We describe the segmentation here and defer the rubric scores to section 3.4.

Performing segmentation and scoring in one call is deliberate, since the boundaries and the credit are mutually informed, and segment boundaries are encouraged to fall where the contribution actually changes. Because the model is a frozen, off-the-shelf LLM, SeRPO needs no labeled trajectory data and no separately trained reward model.

Unlike rule-based splitting on surface signals (app switches, transition keywords), which over-segments, the LLM groups steps by functional purpose, consolidating multi-step error recovery or cross-app authentication into single coherent segments. Crucially, segments are defined by semantic subgoal rather than by surface-string identity of observations, so unlike step-level anchor grouping it does not depend on observations recurring verbatim across rollouts.

3.4. Segment Reward

We now specify the rubric scores that the joint call emits for each segment. Rather than ask the judge for a single score per segment, which conflates distinct notions of quality and is hard to validate, we use a multidimensional, count-

Table 1. The four count-anchored rubric dimensions.

Dimension	Measurement signal
Correctness	Count the error responses in the segment’s outputs and whether its final step succeeded.
Efficiency	Count wasted steps = executed steps – steps the subgoal minimally needs, including redundant or repeated identical calls.
Progress	Trace whether the state the segment produced is consumed by a later segment and advances a goal named in the evaluation.
Exploration fit	Check whether an API-documentation lookup occurred before the segment’s first state-changing action on a previously unseen app.

anchored rubric. For each segment S_m the judge scores four orthogonal dimensions (correctness, efficiency, progress, and exploration fit), each on a 1–5 scale whose anchors are tied to an observable property counted from the segment, and it is required to cite that count in its rationale, making every score auditable. Each dimension and its measurement signal are defined in Table 1. Exploration fit, in particular, targets the dominant premature-action failure mode on AppWorld, in which the agent calls a hallucinated API before consulting its documentation. A dimension that does not apply to a segment’s phase type (e.g. exploration fit for a `login` or `completion` segment) takes the non-penalizing default of 5, so the mean is always over four comparable terms and a segment is never penalized for omitting an action irrelevant to its function.

The scalar contribution reward used downstream is the unweighted mean of the four dimensions,

$$R_m = \frac{1}{4}(d_m^c + d_m^e + d_m^p + d_m^x), \quad d_m^{(\cdot)} \in \{1, \dots, 5\} \quad (1)$$

with each $d_m^{(\cdot)} = \pi_j(S_m, \tau, I)$ produced by the judge given the task instruction I and the full trajectory τ as context. Decomposing the score into explicit, countable sub-criteria grounds each value in observable evidence rather than an unexplained single judgment and, consistent with prior rubric work (Gunjal et al., 2025), reduces the format and length-based hacking to which a single judge score is prone.

A key property is that eq. (1) assigns non-zero partial credit to segments inside failed trajectories. A rollout that fails overall may still contain segments that executed cleanly or recovered effectively, and SeRPO preserves their positive signal while penalizing the segment that introduced the critical error. This is what lets SeRPO extract a learning signal even from groups that every rollout fails the unit tests.

3.5. Segment-Level Policy Optimization

We integrate segment rewards into a GRPO-style objective. The full trajectory is kept as one token sequence for the forward pass, but the advantage of each token is that of its enclosing segment. For each segment we normalize within the rollout group,

$$A_m = \frac{R_m - \mu_{\text{group}}}{\sigma_{\text{group}} + \epsilon}, \quad A(t_i) = A_{\text{seg}(t_i)}, \quad (2)$$

where $\text{seg}(t_i) \in \{1, \dots, M\}$ is the segment index of token t_i , and $\mu_{\text{group}}, \sigma_{\text{group}}$ are the mean and standard deviation of segment rewards across rollouts in the group. The loss is

$$\mathcal{L} = - \sum_{i=1}^N A(t_i) \log \pi_{\theta}(t_i \mid t_{<i}). \quad (3)$$

Tokens in high-scoring segments receive positive advantages and are reinforced, while tokens in low-scoring segments are suppressed. Replacing per-segment advantage in eq. (2) with a single trajectory-level advantage recovers the trajectory-rubric baseline (SeRPO-traj) we ablate in Section 4.3.

Offline GRPO. Standard GRPO is on-policy and requires fresh rollouts at every update. Under our compute budget we adopt an offline variant: a fixed pool of trajectories is collected once from the base policy, segment-level advantages are precomputed, and the policy is updated with eq. (3) using these stored advantages, as in reward-weighted policy optimization. This preserves segment-level credit assignment while removing repeated rollout generation.

4. Experiments

In this section, we empirically demonstrate the following:

- (1) SeRPO yields consistent performance improvements over both trajectory-level and step-level reward assignment;
- (2) the gains stem from segment-level assignment rather than from the rubric alone, which we further explain by analyzing the learned segment rewards;
- (3) SeRPO improves performance using failed trajectories alone, from which binary-outcome methods obtain no learning signal; and
- (4) the learned segmentation is of high quality and yields interpretable rewards.

4.1. Experimental Setup

Benchmarks. We evaluate SeRPO on AppWorld (Trivedi et al., 2024), a benchmark of realistic, long-horizon tasks in which an agent operates nine applications through 457 APIs via free-form code generation. We choose it over the commonly used WebShop (Yao et al., 2022) and ALF-World (Shridhar et al., 2021), where the agent selects from a

Table 2. Main results on AppWorld, broken down by task difficulty. TGC = task goal completion, SGC = scenario goal completion. GiGPO redistributes a binary outcome reward into step-level credit.

Method	Reward Type	Credit Granularity	TGC (%)				SGC (%)			
			Avg.	Easy	Med.	Hard	Avg	Easy	Med.	Hard
Base	–	–	17.9	31.6	12.5	9.5	7.1	15.8	6.2	0.0
GRPO	binary outcome	trajectory-level	18.4	38.6	6.2	9.5	8.9	26.3	0.0	0.0
	fractional outcome	trajectory-level	19.1	31.6	10.4	14.3	8.9	15.8	0.0	9.5
GiGPO	binary outcome	step-level	20.8	43.9	8.3	9.5	8.9	21.1	6.2	0.0
SeRPO	LLM-judge	segment-level	22.0	35.1	12.5	17.5	10.7	21.1	0.0	9.5

small set of templated actions over short horizons; on such bounded benchmarks strong base agents already reach high success rates, leaving little headroom. AppWorld instead exposes a combinatorially open, code-based action space and poses substantially harder, more realistic tasks, on which even the strongest agents succeed only rarely. In this more challenging setting, a large fraction of sampled trajectories naturally end in failure, a regime in which binary-outcome methods provide no fine-grained signal about which steps caused the failure, allowing us to assess whether SeRPO remains effective.

Baselines. We organize our comparison along two axes within the GRPO framework: *reward type* (binary, fractional, or LLM-judge) and *credit granularity* (trajectory, step, or segment).

We consider: (1) the frozen base LLM; (2) GRPO with a binary trajectory-level outcome reward; (3) GRPO with a fractional trajectory-level outcome reward, given by AppWorld’s *task goal completion* score; (4) GiGPO (Feng et al., 2025), which takes binary trajectory-level outcome rewards and redistributes them into fine-grained step-level credit via anchor-state grouping; and (5) SeRPO, which uses an LLM to assign rewards directly at the segment-level.

Training Setup. We use Qwen3.5-9B (Team, 2026) as our base policy, prompted as a ReAct prompt (Yao et al., 2023), and fine-tune it with LoRA adapters under the offline GRPO objective of Section 3.5. For each task we sample a group of $G = 8$ trajectories from the base policy on the AppWorld training split and compute advantages as within-group z -scores. All methods share identical GRPO hyperparameters and differ only in the unit over which the advantage is broadcast: vanilla GRPO uses the full trajectory, whereas SeRPO uses each segment’s token span. The joint segmentation and scoring call uses a single frozen LLM, GPT-5-mini.

GiGPO (Feng et al., 2025) assigns step-level credit by grouping actions taken from the same environment state. Since AppWorld’s code-based action space rarely produces verbatim-recurring states, we define an anchor state by the textual observation preceding each step (API outputs, errors,

or printed results) and group steps by exact string match.

Evaluation Setup. We evaluate on the AppWorld test-normal split and report task goal completion (TGC) and scenario goal completion (SGC). Both metrics are scored programmatically via AppWorld’s state-based unit tests. Due to computational constraints, we report results from a single training seed; we leave a multi-seed analysis of training variance to future work.

4.2. Main Results

Table 2 organizes every arm along the two axes introduced in Section 4.1: *reward type* (binary, fractional, or LLM-judge) and *credit granularity* (trajectory, step, or segment). Reading the table along these two axes isolates what actually drives the gain.

A finer-grained reward value is not enough. Moving from a binary to a fractional (*task goal completion*) outcome reward improves test TGC only marginally (0.7%) and leaves SGC unchanged (8.9%). A denser signal at the same granularity still does not tell the policy *which* phase deserves credit. Density is not the bottleneck.

Step-level credit helps subtasks but not the overall task. GiGPO, which keeps the outcome reward but assigns credit at the step level, improves over both trajectory-level outcome baselines on TGC (20.8 vs. 19.1), confirming that that finer-grained credit assignment helps. Crucially, however, its SGC stays at 8.9, identical to the trajectory-level baselines: step-level credit improves performance on the finer-grained subtasks but does not translate into success on the overall task, which is what SGC measures. Capturing what drives final success requires learning at the level of subgoals, whereas step-level credit is too fine-grained and noisy to represent that structure.

The segment hits the granularity sweet spot. SeRPO assigns credit at the segment level and achieves the best performance on both metrics (22.0 TGC, 10.7 SGC), surpassing GiGPO by 1.2 points in TGC and, more notably,

Table 3. Ablation isolating granularity from the reward, broken down by task difficulty. SeRPO-traj keeps the LLM rubric judge but averages the per-segment scores to a single trajectory-level reward; only the granularity differs from SeRPO.

Method	Reward	Credit Gran.	TGC (%)				SGC (%)			
			Avg.	Easy	Med.	Hard	Avg.	Easy	Med.	Hard
Base	–	–	17.9	31.6	12.5	9.5	7.1	15.8	6.2	0.0
SeRPO-traj	LLM	traj.	17.9	33.3	8.3	11.1	5.4	10.5	0.0	4.8
SeRPO	LLM	seg.	22.0	35.1	12.5	17.5	10.7	21.1	0.0	9.5

by 1.8 points in SGC, the metric on which step-level credit yielded no improvement. Positioned between the coarse trajectory and the fine step, the segment is the only granularity that improves long-horizon scenario completion.

4.3. Ablation Study: Isolating the Effect of Segment-Level Assignment

SeRPO scores each segment with a rubric-based LLM judge and assigns the resulting reward at the segment level. To isolate the effect of segment-level assignment from that of the rubric judge, we conduct an ablation that fixes the judge and varies only the granularity at which its reward is assigned: segment-level assignment (SeRPO) versus trajectory-level assignment (SeRPO-traj). SeRPO-traj uses the *same* rubric judge and the *same* per-segment scores as SeRPO, but averages them into a single trajectory-level reward before training. The two arms thus share an identical reward source and differ only in assignment granularity, isolating the contribution of segment-level assignment.

The same reward helps only at the segment level. SeRPO reaches 22.0/10.7 (TGC/SGC), whereas SeRPO-traj reaches only 17.9/5.4 (See Table 3). Since the two arms use an identical reward and differ only in assignment granularity, the gain is attributable to segment-level *assignment*: the same per-segment scores that are informative when placed on their own segments lose their effect once averaged into a single trajectory-level reward.

Averaging the segment reward yields no gain over base. SeRPO-traj does not exceed the untrained base policy (TGC 17.9 vs. 17.9; SGC 5.4 vs. 7.1). This is not a problem with trajectory-level assignment itself: the trajectory-level outcome baselines in Table 2 do beat base, because a binary outcome reward only ever rewards trajectories that actually succeed. The averaged rubric reward is different. Even a failed trajectory can contain correct early segments (e.g., a clean login, sensible API exploration), so *averaging leaves only a small gap between failure and success* and can even close it within a training group (Details on Figure 3). The reward then pushes the policy toward failed trajectories almost as often as toward successful ones, so the helpful and harmful updates roughly cancel, and the policy never

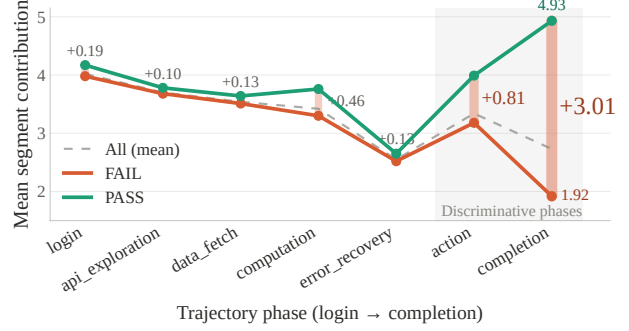


Figure 2. Mean segment contribution by phase type over the SeRPO training trajectories, ordered from login to completion. The PASS–FAIL gap is negligible in early phases and concentrates in action and completion.

improves over base. How a reward should be assigned thus depends on its granularity, and a per-segment signal must stay at the segment level.

4.4. Analysis: Why Segment-Level Assignment Helps

To understand *why* segment-level assignment matters, we analyze the segment contribution scores used during SeRPO training, spanning 720 trajectories across 90 task groups.

The reward survives even when the binary outcome is silent. The central failure mode of binary GRPO is that when all rollouts in a group fail, the reward variance is zero and the normalized advantage is identically zero, leaving no learning signal. We find 26 such all-fail groups (all $G=8$ rollouts fail the unit tests). In every one of them, the segment contribution scores retain non-zero variance, with a pooled within-group standard deviation of 1.37 on the 1–5 scale. Where the binary reward provides no gradient, the segment reward still does.

Only a few phases separate success from failure, so where the reward is assigned matters. Figure 2 breaks the contribution scores down by phase type, split by trajectory outcome. The discriminative signal is highly concentrated: *completion* segments separate success from failure almost perfectly (scoring 4.93 on passing trajectories versus 1.92 on failing ones, a gap of roughly three points), and *action* segments carry a secondary gap of about 0.8 points. The early phases (*login*, *api_exploration*, *data_fetch*) score nearly the same whether the trajectory ultimately passes or fails, differing by at most 0.2 points, because successful and failed rollouts explore similarly and the failure surfaces only later.

This breakdown explains two observations at once. First, it explains why a single trajectory-averaged score is too weak: the discriminative *completion* segment is only

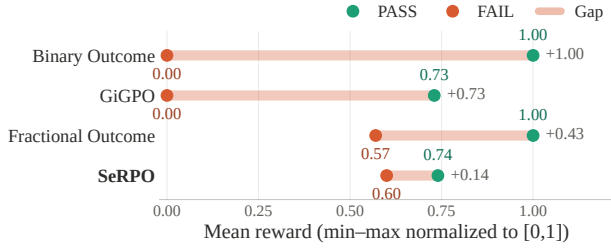


Figure 3. Mean trajectory-level reward by outcome (min-max normalized to $[0, 1]$). Binary and GiGPO collapse FAIL reward to 0 (no signal on failures), whereas SeRPO keeps it stays well above the floor (0.60), giving a dense reward even on failed trajectories.

one of many segments in a trajectory, so averaging dilutes its signal across the early phases, directly accounting for the failure of SeRPO-traj in Table 3. Second, it explains why segment-level assignment is essential: within a single failed trajectory, SeRPO can reward genuinely good exploration (e.g., `api_exploration`, which scores 3.68) while penalizing a botched segment (e.g., `completion`, which scores 1.92), a distinction that neither an outcome reward nor a trajectory-averaged reward can represent.

The reward credits good process, even in failures. Because the reward is assigned per segment, it can discriminate within a failed trajectory: well-executed segments still earn high scores while the segment responsible for the failure is penalized, rather than the whole trajectory being uniformly zeroed out. Figure 3 bears this out: the three outcome-derived rewards separate successful from failed trajectories strongly, whereas SeRPO’s averaged segment reward separates them only weakly (0.14), since failed trajectories still score well on their correct segments.

SeRPO’s reward is therefore not a relabeled outcome but a process signal, with two consequences. First, it is why averaging fails: a 0.14 separation is too weak to drive learning once collapsed to a trajectory mean, as shown in the ablation study in Section 4.3. Second, SeRPO alone stays well above zero on failed trajectories, whereas binary and GiGPO both vanish whenever the outcome is zero; its outcome-independent scoring thus retains a usable signal even when every rollout fails, as we demonstrate in Section 4.5.

4.5. Training on Fail-Only Trajectories

A direct test of the claim that the bottleneck is reward *structure*, not density, is to train only on trajectories that fail the unit tests, where the binary outcome carries no signal. We compare four rewards on this fail-only split (Table 4).

By construction, a binary outcome reward provides *no* gradient here: with every rollout scored 0, the group mean and variance are both zero and the normalized advantage

Table 4. Training on fail-only trajectories, broken down by task difficulty. Outcome-based rewards (binary, fractional, GiGPO) have little or no gradient where every rollout scores zero. SeRPO’s outcome-independent segment scores retain a signal throughout.

Method	TGC (%)				SGC (%)			
	Avg.	Easy	Med.	Hard	Avg.	Easy	Med.	Hard
Base	17.9	31.6	12.5	9.5	7.1	15.8	6.2	0.0
GRPO (bin.)	No Learning Signal							
GiGPO	No Learning Signal							
GRPO (frac.)	18.4	35.1	8.3	11.1	5.4	15.8	0.0	0.0
SeRPO	20.2	36.8	12.5	11.1	10.7	21.1	6.2	4.8

is identically zero, so binary GRPO cannot learn at all and is reported as “no learning signal” rather than measured. GiGPO, which in our experiments uses the binary outcome, inherits this collapse: both its episode-level advantage and its anchor-group (step-level) advantage are derived from the same uniformly-zero outcome, so even its fine-grained step signal vanishes on the fail-only split. A step-level method does not escape the problem when the underlying reward is outcome-based.

A fractional (task-goal-completion) reward survives only where rollouts differ in partial completion. It trains, but barely improves over the base policy on TGC (18.4 vs. 17.9) and *degrades* SGC (5.4 vs. 7.1): on the subset where all rollouts pass zero tests its advantage collapses to zero just as the binary one does, so the gradient it does receive is too weak and too uneven to lift full-scenario completion. SeRPO’s segment scores, by contrast, are independent of the outcome and retain non-zero variance even on all-fail groups, giving it a usable gradient throughout. It is the only reward that improves over base on both metrics using failed data alone, reaching 20.2/10.7 (TGC/SGC) and raising scenario completion from 7.1 to 10.7. This is the cleanest form of our central claim: where outcome-based rewards have nothing to learn from, the segment abstraction, not a denser signal, is what keeps learning alive.

4.6. Case Study: Interpretability, Faithfulness, and Stability of the Reward

Figure 4 contrasts step-level and segment-level credit on one successful trajectory: rescheduling a sleep alarm and disabling the others, which requires recovering from a missing API error, authenticating through several failed login attempts, and finally querying and updating the alarms. Two failure modes of step-level credit are visible.

First, step-level credit is *unfaithful* to task progress: it rewards actions for incidental reasons rather than for advancing the task. GiGPO scores the failed `show_alarms` retry at step 14 above the successful login that immediately precedes it, because its bonus attaches to steps whose preceding

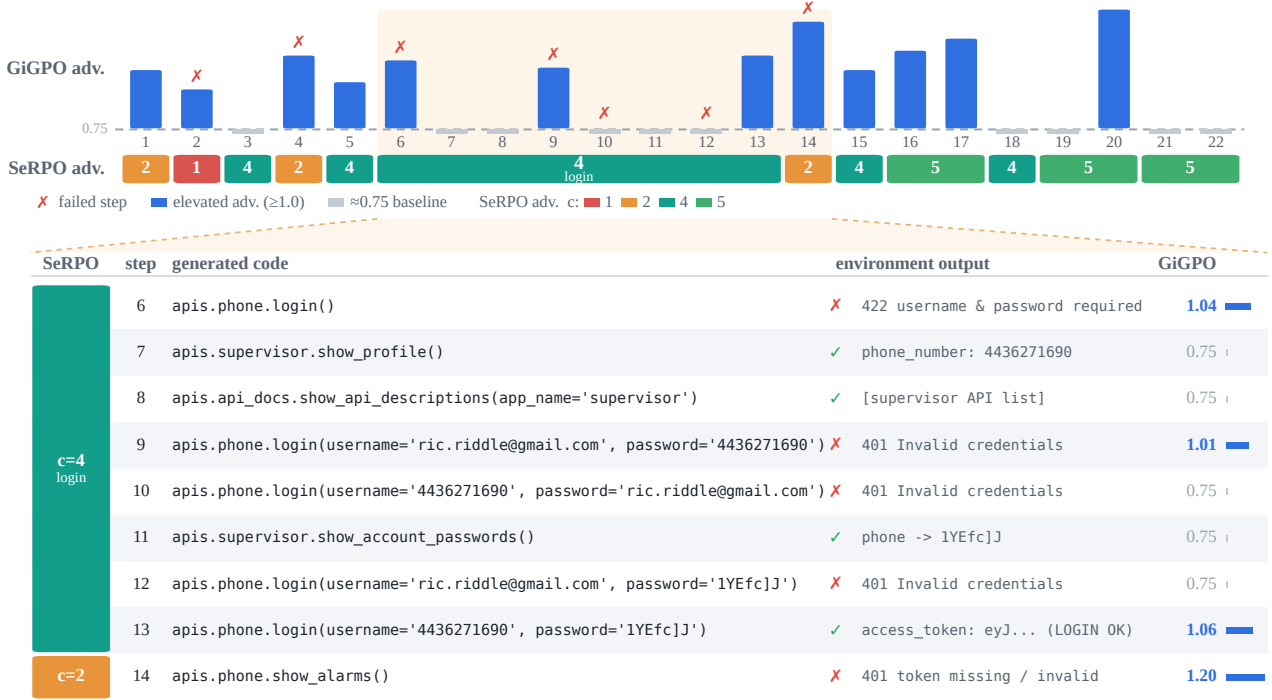


Figure 4. **GiGPO step reward vs. SeRPO segment reward.** GiGPO ranks a failed login (step 6, 1.04) on par with the successful one (step 13, 1.06), and gives the failed 401 retry (step 14) the window’s highest score(1.20); its bonus tracks anchor recurrence and depth, not task success. **SeRPO** instead wraps steps 6–13 into a single subgoal, while the wasteful retry (step 14) forms its own low-credit segment.

observation happens to recur across rollouts. SeRPO instead places the same retry in its own low-value segment, below the segment that achieves authentication.

Second, the step is too fine a unit to carry a *stable* signal. Within a single coherent subgoal, GiGPO’s per-step credit scatters: two steps that together complete the alarm update receive very different scores, one treated as highly advantageous while the other sits at the baseline, despite their identical role. SeRPO instead scores the subgoal as a whole, assigning the segment one consistent value.

The contrast supports all three properties. SeRPO’s reward is *faithful*: scoring a coherent subgoal as a unit ties credit to whether that subgoal was achieved, rather than to which states happen to recur. It is *stable*: aggregating to the subgoal level removes the step-level variance that makes per-step credit scatter across actions of identical role. And it is *interpretable*: each segment’s score corresponds to a subgoal, so the reward reveals which parts of a trajectory succeeded and which failed. This makes reward hacking localizable: a segment scoring high without completing its subgoal points directly to where credit was misassigned, which a single trajectory-level score is too coarse to attribute and step-level credit too fine to tie to a meaningful unit.

5. Conclusion

We presented SeRPO, in which a single frozen-LLM call jointly segments a trajectory and scores each segment with a multidimensional, count-anchored rubric, assigning partial credit at the granularity of subgoal-aligned segments and injecting it into a GRPO objective with no separately trained reward model and no labeled trajectory data. Framing the design question as the *granularity* at which credit is assigned rather than the density of the signal, we showed on AppWorld that the segment, an intermediate granularity between the sparse outcome reward and the noisy step reward, outperforms trajectory- and step-level baselines on both metrics, and that a controlled ablation attributes the gain to segment-level *assignment* rather than to the rubric itself. Because each segment is scored independently of the outcome, SeRPO also keeps a usable gradient where outcome-based rewards have none, improving over base when trained on failed trajectories alone; and because each segment’s score corresponds to a subgoal, the reward is interpretable, localizing where credit was earned or might be exploited. We leave extending SeRPO to on-policy training and other long-horizon benchmarks to future work.

References

- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Ding, L. AdaRubric: Task-adaptive rubrics for reliable LLM agent evaluation and reward learning. *arXiv preprint arXiv:2603.21362*, 2026.
- Feng, L., Xue, Z., Liu, T., and An, B. Group-in-group policy optimization for LLM agent training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Gunjal, A., Wang, A., Lau, E., Nath, V., He, Y., Liu, B., and Hendryx, S. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*, 2025.
- He, S., Feng, L., Wei, Q., Cheng, X., Feng, L., and An, B. Hierarchy-of-groups policy optimization for long-horizon agentic tasks. In *International Conference on Learning Representations (ICLR)*, 2026.
- Li, J., Wang, Y., Yan, D., Tian, Y., Xu, Z., Song, H., Xu, P., and Cheong, L. L. Salt: Step-level advantage assignment for long-horizon agents via trajectory graph. In *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 4709–4725, 2026.
- Peng, J., Liu, Y., Zhou, R., Fleming, C., Wang, Z., Garcia, A., and Hong, M. HiPER: Hierarchical reinforcement learning with explicit credit assignment for large language model agents. *arXiv preprint arXiv:2602.16165*, 2026.
- Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., and Hausknecht, M. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021.
- Team, Q. Qwen3.5-omni technical report. 2026.
- Trivedi, H., Khot, T., Hartmann, M., Manku, R., Dong, V., Li, E., Gupta, S., Sabharwal, A., and Balasubramanian, N. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, 2024.
- Yao, S., Chen, H., Yang, J., and Narasimhan, K. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 20744–20757, 2022.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Zhang, Y., Jiang, S., Li, R., Tu, J., Su, Y., Deng, L., Guo, X., Lv, C., and Lin, J. DeepPlanning: Benchmarking long-horizon agentic planning with verifiable constraints. *arXiv preprint arXiv:2601.18137*, 2026.